

Laporan Hands-on 5

Thread synchronization dibahas secara lebih mendalam setelah dasar concurrency dikenalkan sebelumnya. Fokus utamanya adalah bagaimana beberapa thread harus **berkoordinasi** saat mengakses shared data atau saat saling menunggu kondisi tertentu. Mekanisme yang dipakai meliputi **mutex**, **semaphore**, **condition variable**, serta pola klasik seperti **producer-consumer** dan **readers-writers**.

Seluruh eksperimen memperlihatkan bahwa concurrency yang benar tidak cukup hanya dengan menjalankan banyak thread sekaligus. Program concurrent juga harus memastikan bahwa akses terhadap shared resource tetap **aman**, urutan eksekusi yang penting tetap **terkendali**, dan thread tidak membuang waktu CPU dengan polling yang tidak perlu.

Section 1 - Membuat Banyak Thread dan Mengamati Eksekusi Konkuren

Bentuk paling awal dari multithreading di dalam satu process terlihat melalui beberapa thread worker yang masing-masing menjalankan pekerjaan serupa secara independen. Tujuan utamanya adalah menunjukkan bahwa sebuah process dapat memiliki lebih dari satu jalur eksekusi dan bahwa hasil keluaran dari thread-thread tersebut dapat saling terinterleaving sesuai keputusan scheduler. Tahap awal ini menjadi dasar sebelum membahas shared data dan sinkronisasi yang lebih rumit.

Implementasi menggunakan tiga thread yang masing-masing menerima id berbeda dan mencetak pesan "Thread %d is running: step %d" sebanyak lima kali. Setelah setiap iterasi, thread melakukan `usleep(100000)` agar perbedaan interleaving lebih mudah diamati. Main thread membuat seluruh worker dengan `pthread_create`, lalu menunggu semuanya selesai menggunakan `pthread_join`. Karena tidak ada shared mutable data yang dimodifikasi bersama, eksperimen ini aman walaupun output-nya dapat muncul dalam urutan yang berubah-ubah.

Dari sudut pandang sistem operasi, pembahasan ini menegaskan bahwa thread-thread di dalam satu process memiliki **execution path** masing-masing walaupun tetap berbagi address space dan resource process yang sama. Ketidakteraturan urutan output bukanlah bug, melainkan konsekuensi normal dari scheduler yang menjalankan thread sesuai keadaan runtime. Dengan demikian, mahasiswa dapat melihat bahwa concurrency sejak awal sudah memperkenalkan **nondeterministic ordering**.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab1_basic_threads.c
Jumlah thread	3 worker thread
Iterasi per thread	5 langkah
Fungsi sinkronisasi	pthread_join
Shared data yang dimodifikasi	Tidak ada

Komponen	Informasi
Fenomena utama	Output antar thread saling terinterleaving
Konsep utama	Basic multithreading dan nondeterministic ordering

Kesimpulan Section 1

Satu process dapat menjalankan banyak thread secara bersamaan tanpa harus membuat banyak process terpisah. Walaupun hasilnya tidak selalu muncul dalam urutan yang sama, hal tersebut justru menunjukkan bahwa thread benar-benar dieksekusi secara concurrent. Pola seperti ini menjadi fondasi penting untuk memahami eksperimen sinkronisasi berikutnya.

Section 2 - Race Condition pada Shared Data

Shared memory memberi kemudahan komunikasi antarthread, tetapi juga membuka peluang terjadinya **race condition**. Dua thread secara bersamaan menjalankan fungsi increment terhadap variabel global counter sebanyak 1000000 kali. Secara matematis, hasil akhir seharusnya 2000000, tetapi nilai aktual yang diamati sering kali lebih kecil. Perbedaan ini menjadi bukti bahwa akses concurrent terhadap shared variable tanpa proteksi dapat merusak correctness program.

Masalah utamanya terletak pada operasi counter++ yang terlihat sederhana tetapi sebenarnya bukan operasi atomik. Untuk menambah satu nilai counter, prosesor harus membaca nilai lama, menambah satu, lalu menulis hasil baru. Bila dua thread melakukan rangkaian langkah itu pada waktu yang bertumpang tindih, salah satu update dapat hilang. Fenomena ini menunjukkan bahwa kebenaran concurrent program tidak hanya ditentukan oleh source code, tetapi juga oleh urutan runtime yang terjadi saat instruksi dieksekusi.

Alasan mengapa shared memory yang efisien justru dapat menjadi sumber kesalahan paling serius dalam multithreading tampak jelas di sini. Race condition tidak selalu mudah dikenali dari pembacaan kode saja, sebab kode dapat terlihat benar secara sequential. Namun ketika dijalankan secara concurrent, hasil aktual membuktikan bahwa program membutuhkan mekanisme mutual exclusion untuk melindungi shared resource.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab2_race_condition.c
Shared resource	Variabel global counter
Jumlah thread	2 thread
Iterasi per thread	1000000 kali
Nilai yang diharapkan	2000000
Hasil aktual	Sering lebih kecil dari nilai harapan
Konsep utama	Race condition dan lost update

Kesimpulan Section 2

Shared variable tidak dapat diakses secara concurrent tanpa proteksi apabila correctness hasil ingin dijaga. Walaupun setiap thread hanya melakukan increment sederhana, hasil akhir tetap dapat salah karena update saling menimpa. Kondisi tersebut menegaskan bahwa multithreading selalu memerlukan perhatian khusus ketika ada shared mutable data.

Section 3 - Menyelesaikan Race Condition dengan Mutual Exclusion

Perbaikan terhadap masalah pada pengujian sebelumnya dilakukan dengan menambahkan **mutex** untuk melindungi critical section. Setiap thread harus memperoleh lock sebelum menaikkan nilai counter, lalu melepaskannya segera setelah update selesai. Dengan demikian, operasi counter++ yang semula rawan interleaving dapat diperlakukan secara eksklusif dari sudut pandang thread lain.

Hasil eksperimen menunjukkan bahwa nilai aktual counter kini selalu sesuai dengan nilai yang diharapkan, yaitu 2000000. Stabilitas ini membuktikan bahwa mutex berhasil menghilangkan lost update dan memulihkan correctness program. Dengan kata lain, sinkronisasi tidak mempercepat program, tetapi membuat program concurrent kembali menghasilkan jawaban yang **benar**.

Trade-off penting pada sistem concurrent juga terlihat jelas. Ketika critical section dilindungi oleh mutex, dua thread tidak lagi bebas mengupdate counter secara bersamaan. Artinya, sinkronisasi mengurangi interleaving di area sensitif agar shared data tetap aman. Hal ini memperlihatkan prinsip desain yang sangat penting dalam sistem operasi: **semakin kecil critical section, semakin kecil pula biaya sinkronisasi yang harus dibayar**.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab3_mutex_counter.c
Mekanisme sinkronisasi	pthread_mutex_t lock
Shared resource	Variabel global counter
Jumlah thread	2 thread
Nilai yang diharapkan	2000000
Nilai aktual	Konsisten sesuai ekspektasi
Konsep utama	Mutual exclusion dan critical section

Kesimpulan Section 3

Mutual exclusion dengan mutex merupakan solusi langsung dan efektif untuk race condition pada shared variable. Dengan membatasi akses ke critical section, hasil program menjadi stabil dan sesuai dengan perhitungan matematis. Kebutuhan akan sinkronisasi di titik ini bukan lagi tambahan opsional, melainkan mekanisme wajib ketika thread berbagi data yang dapat diubah.

Section 4 - Producer-Consumer dengan Sinkronisasi Semaphore

Pola klasik **producer-consumer** diperkenalkan menggunakan kombinasi semaphore dan mutex. Satu thread producer menaruh item ke buffer, sedangkan satu thread consumer mengambil item dari buffer yang sama. Buffer yang dipakai hanya berukuran satu slot, sehingga producer dan consumer harus berkoordinasi agar producer tidak menimpa data yang belum diambil dan consumer tidak membaca data yang belum tersedia.

Mekanisme koordinasi dibangun dengan dua semaphore, yaitu `empty` dan `full`. Semaphore `empty` menyatakan bahwa buffer kosong dan producer boleh menulis, sedangkan semaphore `full` menyatakan bahwa ada item siap dibaca consumer. Di luar itu, mutex digunakan untuk melindungi operasi akses aktual ke buffer sehingga proses baca dan tulis tidak saling bertabrakan. Dengan demikian, praktikum ini menunjukkan bahwa sinkronisasi tidak hanya tentang mutual exclusion, tetapi juga tentang **ordering**.

Hasil yang diharapkan adalah setiap item yang diproduksi akan dikonsumsi tepat satu kali. Producer tidak akan melanjutkan produksi bila buffer belum kosong, dan consumer tidak akan melanjutkan konsumsi bila belum ada item tersedia. Dari sudut pandang sistem operasi, lab ini menunjukkan bahwa semaphore dapat dipakai untuk mengatur **ketersediaan resource**, sedangkan mutex melindungi area kritis yang benar-benar menyentuh data.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>lab4_producer_consumer.c</code>
Pola sinkronisasi	Producer-consumer
Shared resource	Buffer ukuran satu
Semaphore	<code>empty</code> dan <code>full</code>
Mutex	<code>lock</code>
Hasil yang diharapkan	Setiap item diproduksi dan dikonsumsi tepat sekali
Konsep utama	Ordering dengan semaphore dan proteksi data dengan mutex

Kesimpulan Section 4

Sinkronisasi concurrent program sering membutuhkan lebih dari satu mekanisme. Semaphore memastikan producer dan consumer berjalan dalam urutan yang benar, sedangkan mutex menjaga agar akses aktual ke buffer tetap aman. Praktikum ini memperjelas perbedaan antara **koordinasi urutan eksekusi** dan **mutual exclusion**.

Section 5 - Blocking dan Unblocking Thread dengan Condition Variable

Sinkronisasi yang lebih efisien dibanding polling muncul melalui **condition variable**. Satu thread worker memeriksa apakah data sudah siap. Jika belum, worker tidak terus-menerus memeriksa secara aktif, melainkan masuk ke keadaan blocked melalui `pthread_cond_wait`.

Thread controller kemudian menyiapkan data, mengubah kondisi, dan membangunkan worker dengan `pthread_cond_signal`.

Hasil eksperimen menunjukkan bahwa worker mula-mula mencetak pesan bahwa data belum siap dan bahwa ia memasuki keadaan blocking. Setelah sekitar tiga detik, controller menyiapkan `shared_data = 42`, mengubah flag `data_ready`, lalu memberikan sinyal. Worker kemudian bangun dan melanjutkan eksekusi menggunakan data yang sudah tersedia. Alur ini menunjukkan bahwa thread dapat menunggu suatu kondisi tanpa membuang CPU untuk busy waiting.

Dari perspektif sistem operasi, condition variable memperlihatkan hubungan langsung antara **state transition** thread dan sinkronisasi. Thread worker berpindah dari running ke blocked hingga ada event yang membangunkannya. Dengan demikian, pembahasan ini tidak hanya membahas correctness, tetapi juga **efisiensi pemakaian CPU** dalam concurrent programming.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab5_blocking_condition.c
Shared state	data_ready dan shared_data
Mekanisme utama	pthread_cond_wait dan pthread_cond_signal
Thread yang menunggu	Worker
Thread yang memberi sinyal	Controller
Data akhir	shared_data = 42
Konsep utama	Blocking synchronization dengan condition variable

Kesimpulan Section 5

Condition variable memungkinkan thread menunggu secara efisien sampai suatu syarat benar-benar terpenuhi. Berbeda dengan polling, thread tidak membuang waktu CPU untuk mengecek kondisi berulang kali. Pemahaman tentang sinkronisasi yang baik di sini bukan hanya menjaga correctness, tetapi juga mengoptimalkan penggunaan resource sistem.

Section 6 - Implementasi Critical Section pada Shared Bank Account

Konsep critical section berubah menjadi simulasi yang lebih realistis, yaitu update saldo rekening bank bersama oleh banyak thread. Setiap thread membaca saldo lama, menunggu sebentar untuk memperjelas timing, lalu menuliskan saldo baru setelah ditambah `DEPOSIT_AMOUNT`. Jika bagian sensitif ini tidak diproteksi, dua thread dapat membaca saldo lama yang sama dan menimpa update satu sama lain. Karena itu, operasi baca-hitung-tulis ditempatkan di dalam critical section yang dilindungi mutex.

Kekuatan lab ini terletak pada visualisasi yang sangat jelas. Setiap thread mencetak saldo yang dibaca, lalu mencetak saldo baru setelah update. Dengan adanya `sleep(1)` di tengah critical section, mahasiswa dapat melihat bahwa tanpa proteksi keadaan ini akan sangat rawan race condition. Ketika mutex dipakai dengan benar, thread-thread tetap dapat

berjalan secara concurrent, tetapi update saldo hanya dilakukan satu per satu di area yang sensitif.

Hasil akhirnya juga mudah diverifikasi secara matematis. Dengan `THREADS = 5`, `DEPOSITS_PER_THREAD = 3`, dan `DEPOSIT_AMOUNT = 100`, maka saldo akhir yang diharapkan adalah **1500**. Ketika mutex digunakan, hasil aktual memang sesuai. Ini memperlihatkan bahwa critical section bukanlah seluruh program, melainkan hanya bagian kecil yang benar-benar menyentuh shared resource.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>critical_section_bank.c</code>
Shared resource	Variabel balance
Jumlah thread	5 thread
Deposit per thread	3 kali
Nilai deposit	100
Saldo akhir yang diharapkan	1500
Konsep utama	Identifikasi dan proteksi critical section

Kesimpulan Section 6

Critical section adalah bagian spesifik dari program yang harus dijaga dari akses concurrent. Dengan melindungi operasi update saldo menggunakan mutex, hasil akhir menjadi konsisten dan sesuai perhitungan. Contoh ini sangat efektif untuk menunjukkan hubungan langsung antara konsep abstrak critical section dan kasus nyata pada shared financial data.

Section 7 - Mengamati Race Condition Secara Langsung

Race condition kembali diangkat, tetapi dalam bentuk yang lebih singkat dan langsung. Dua thread menjalankan increment terhadap shared counter sebanyak 1000000 kali tanpa proteksi. Pengamatan ini berfungsi sebagai penguatan konsep bahwa race condition bukan kasus khusus pada satu program tertentu, melainkan pola umum yang akan terus muncul ketika shared data diakses secara concurrent tanpa sinkronisasi.

Keunggulan eksperimen ini adalah kesederhanaan kodenya. Karena source code sangat pendek, mahasiswa dapat langsung melihat bahwa bug bukan disebabkan oleh struktur program yang rumit. Justru pada program yang sangat sederhana ini, hasil aktual yang lebih kecil dari nilai harapan menjadi bukti kuat bahwa masalah concurrency bersumber dari interleaving runtime, bukan dari panjang atau kerumitan kode.

Dari sisi pembelajaran, eksperimen ini berfungsi sebagai jembatan menuju pembahasan berikutnya yang memperkenalkan solusi menggunakan mutex dan semaphore. Dengan mengamati race condition dalam bentuk paling ringkas, mahasiswa dapat lebih mudah memahami mengapa mekanisme mutual exclusion menjadi kebutuhan mendasar dalam concurrent programming.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab1_race.c
Shared resource	Variabel counter
Jumlah thread	2 thread
Iterasi per thread	1000000 kali
Nilai yang diharapkan	ITER * 2
Hasil aktual	Sering lebih kecil dari ekspektasi
Konsep utama	Race condition sederhana pada shared counter

Kesimpulan Section 7

Race condition adalah konsekuensi alami dari shared memory tanpa proteksi. Bahkan program yang sangat kecil dan mudah dibaca pun dapat menghasilkan nilai salah jika dua thread melakukan update pada data yang sama secara bersamaan. Hal tersebut memperjelas bahwa masalah concurrency harus dianalisis pada level runtime, bukan hanya dari tampilan source code.

Section 8 - Implementasi Mutual Exclusion dengan Mutex

Solusi langsung terhadap race condition pada pengujian sebelumnya diberikan melalui **mutex** sebagai lock. Sebelum thread mengubah nilai counter, ia harus mengambil mutex, melakukan increment, lalu melepaskannya kembali. Dengan pola tersebut, critical section menjadi eksklusif dan tidak dapat dimasuki dua thread secara bersamaan.

Hal terpenting dari praktikum ini adalah kestabilan hasil akhir. Nilai Actual selalu sama dengan nilai Expected, sehingga mahasiswa dapat membandingkan secara langsung perilaku program sebelum dan sesudah sinkronisasi. Perubahan ini menjadi bukti konkret bahwa mutex tidak mengubah tujuan perhitungan, tetapi mengubah **cara akses shared resource** agar tidak terjadi lost update.

Mutex juga menegaskan posisinya sebagai mekanisme mutual exclusion yang paling umum dalam pemrograman thread. Ia sederhana, langsung, dan sangat cocok ketika tujuannya adalah memastikan bahwa hanya satu thread yang masuk ke critical section pada satu waktu. Landasan seperti ini penting sebelum beralih ke semaphore yang memiliki kemampuan lebih luas.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab2_mutex.c
Mekanisme utama	pthread_mutex_lock dan pthread_mutex_unlock
Shared resource	Variabel counter
Jumlah thread	2 thread
Hasil akhir	Selalu sesuai dengan ekspektasi

Komponen	Informasi
Konsep utama	Mutual exclusion dengan mutex

Kesimpulan Section 8

Mutex memberikan cara yang sederhana dan efektif untuk menghilangkan race condition. Dengan memaksa hanya satu thread mengakses critical section pada satu waktu, hasil perhitungan kembali menjadi benar dan dapat diprediksi. Contoh ini menjadi penjelasan paling jelas tentang bagaimana sinkronisasi memulihkan correctness pada shared data.

Section 9 - Mutual Exclusion Berbasis Semaphore

Pemahaman mutual exclusion diperluas dengan mengganti mutex menggunakan **binary semaphore**. Secara perilaku, semaphore dengan nilai awal 1 dapat bertindak seperti lock: `sem_wait` menurunkan nilai dan memblok thread jika semaphore bernilai 0, sedangkan `sem_post` menaikkannya kembali agar thread lain dapat masuk. Dengan cara ini, semaphore juga dapat digunakan untuk melindungi critical section.

Dari hasil program, nilai counter kembali benar dan konsisten, sama seperti pada penggunaan mutex. Ini menunjukkan bahwa baik mutex maupun semaphore dapat menyelesaikan race condition pada shared counter. Namun secara konsep, semaphore lebih umum karena dapat digunakan tidak hanya untuk mutual exclusion, tetapi juga untuk sinkronisasi ketersediaan resource atau pengaturan ordering seperti pada producer-consumer.

Perbandingan ini penting bagi mahasiswa karena memperlihatkan bahwa beberapa mekanisme sinkronisasi dapat menghasilkan efek correctness yang sama, tetapi memiliki domain penggunaan yang berbeda. Mutex lebih spesifik untuk ownership lock pada thread, sedangkan semaphore lebih fleksibel untuk masalah koordinasi yang lebih luas.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab3_semaphore.c
Mekanisme utama	Binary semaphore (<code>sem_wait</code> , <code>sem_post</code>)
Shared resource	Variabel counter
Jumlah thread	2 thread
Nilai akhir	Benar dan konsisten
Konsep utama	Mutual exclusion dengan semaphore

Kesimpulan Section 9

Semaphore dapat digunakan untuk menegakkan mutual exclusion dengan hasil yang sama benarnya seperti mutex. Namun, semaphore bersifat lebih umum dan karenanya penting dipahami sebagai mekanisme sinkronisasi yang tidak terbatas pada lock sederhana saja. Pandangan terhadap ragam alat sinkronisasi dalam sistem operasi pun menjadi lebih luas.

Section 10 - Producer-Consumer Synchronization

Pola producer-consumer kembali digunakan, tetapi dalam bentuk yang lebih ringkas dan fokus pada ide bahwa sinkronisasi bukan sekadar proteksi shared data. Producer menghasilkan angka 1 sampai 5, sedangkan consumer mengonsumsi data dari buffer yang sama. Dengan kombinasi semaphore `empty`, `full`, dan mutex lock, sistem memastikan bahwa producer tidak menulis ketika buffer belum kosong dan consumer tidak membaca ketika buffer belum berisi.

Berbeda dari race condition biasa, masalah utama pada pola ini adalah **availability and ordering**. Data yang belum tersedia tidak boleh dikonsumsi, dan data yang belum dikonsumsi tidak boleh ditimpa. Oleh karena itu, semaphore memegang peran sentral untuk mengatur kapan satu thread boleh melanjutkan pekerjaan. Mutex tetap dibutuhkan, tetapi lebih berfungsi sebagai pelindung akses buffer yang sesaat, bukan sebagai pengatur keseluruhan urutan eksekusi.

Hasil yang diamati adalah alternasi ketat antara produksi dan konsumsi. Setiap nilai yang diproduksi akan diambil sebelum producer menaruh nilai berikutnya. Dengan demikian, eksperimen ini memperlihatkan bahwa sinkronisasi concurrent program sering melibatkan dua dimensi sekaligus: **safety** dan **coordination**.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	<code>lab4_producer_consumer.c</code>
Pola	Producer-consumer
Semaphore	<code>empty</code> dan <code>full</code>
Proteksi buffer	<code>pthread_mutex_t lock</code>
Jumlah item	5 item
Hasil utama	Produksi dan konsumsi terjadi bergantian secara aman
Konsep utama	Sinkronisasi ketersediaan data

Kesimpulan Section 10

Sinkronisasi concurrent program tidak selalu berpusat pada perbaikan race condition. Dalam pola producer-consumer, masalah utamanya adalah kapan thread boleh melanjutkan kerja berdasarkan keadaan buffer. Praktikum ini memperjelas bahwa semaphore sangat efektif untuk mengatur ketersediaan resource, sedangkan mutex melindungi akses sesaat terhadap shared data.

Section 11 - Readers-Writers Problem

Pola klasik **readers-writers** menutup rangkaian bahasan ini, yaitu situasi ketika banyak reader boleh mengakses data secara bersamaan, tetapi writer membutuhkan akses eksklusif. Pola tersebut penting karena memperlihatkan bahwa sinkronisasi tidak selalu berarti “hanya satu thread boleh masuk”. Dalam kasus tertentu, sistem justru diinginkan

membiarkan banyak thread berjalan bersamaan selama mereka hanya membaca dan tidak mengubah data.

Implementasi menggunakan `read_count`, `mutex` `mutex`, dan `mutex` `write_lock`. Reader pertama yang masuk akan mengunci `write_lock` agar writer tidak dapat masuk selama masih ada reader aktif. Reader berikutnya boleh tetap membaca tanpa saling memblok. Ketika reader terakhir keluar, `write_lock` dilepas sehingga writer dapat melanjutkan. Sementara itu, writer selalu harus memperoleh `write_lock` secara eksklusif sebelum menulis.

Hasil yang diharapkan adalah beberapa reader dapat berjalan bersama, sedangkan writer berjalan sendirian. Desain sinkronisasi pada pola ini dapat dioptimalkan untuk mempertahankan concurrency sebanyak mungkin tanpa mengorbankan safety. Dibandingkan mutual exclusion biasa, pola ini lebih kaya karena memperhitungkan **jenis akses** terhadap shared resource, bukan hanya keberadaan akses itu sendiri.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab5_readers_writers.c
Shared state	read_count
Lock utama	mutex dan write_lock
Akses reader	Boleh concurrent
Akses writer	Harus eksklusif
Hasil utama	Controlled concurrency antara reader dan writer
Konsep utama	Readers-writers synchronization

Kesimpulan Section 11

Concurrency yang aman tidak selalu identik dengan serialisasi penuh. Dalam pola readers-writers, sistem masih dapat mengizinkan banyak reader bekerja bersamaan asalkan writer tetap memperoleh eksklusivitas saat diperlukan. Penutup ini memperlihatkan bahwa sinkronisasi pada sistem operasi sering kali perlu dirancang sesuai sifat akses terhadap resource.

Penutup

Dari seluruh eksperimen yang dijalankan, dapat disimpulkan bahwa sinkronisasi merupakan elemen inti dalam concurrent programming. Rangkaian bahasan ini memperlihatkan bagaimana thread-thread yang berjalan bersamaan harus diatur agar tidak merusak shared data, tidak menunggu secara tidak efisien, dan tidak berjalan pada urutan yang salah. Dari mutex, semaphore, condition variable, producer-consumer, hingga readers-writers, setiap mekanisme menunjukkan fungsi yang berbeda tetapi saling melengkapi.

Secara keseluruhan, rangkaian praktikum ini menegaskan bahwa concurrency yang baik tidak semata-mata bertujuan menjalankan banyak thread sekaligus, tetapi juga

memastikan bahwa hasil program tetap **benar**, resource digunakan secara **aman**, dan CPU tidak terbuang untuk aktivitas menunggu yang tidak efisien. Dengan demikian, praktikum ini memberi landasan yang kuat untuk memahami topik lanjutan seperti deadlock, starvation, resource ordering, dan blocking synchronization.